

ROGUEOS™ — ASYNCHRONOUS GOVERNED AGENT / “ALFRED” RUNTIME

Final Architecture & Build Framework — v0.1

Rogue Management Group, LLC • August 27, 2026 • Private Working Specification

ROGUEOS™ — ASYNCHRONOUS GOVERNED AGENT / “ALFRED” RUNTIME

Final Architecture & Build Framework — v0.1

Owner: Rogue Management Group, LLC

Status: Architecture / prototype build specification

Date: August 27, 2026

1. Core proposition

The agent may continue working when the human is away; authority does not leave with the agent.

The agent is a worker. The governance plane is the authority boundary. A model may propose or select actions, but it does not decide whether it remains authorized to operate.

2. Architecture

HUMAN PRINCIPAL

|

v

BUILDER'S PERMIT / AUTHORITY

|

v

GOVERNANCE ENGINE

|---- ALLOW

|---- DENY

|---- ESCALATE

|---- CONTAIN

|

v

ASYNCH AGENT RUNTIME

|---- local model / model adapter

|---- task queue

|---- controlled tools

|---- persistent state

|

v

WATCHDOG

|

+---- deviation ----> INDEPENDENT FAILSAFE

| |

| v

| REVOKE

| CONTAIN

|

v

EXECUTION

|

v

EVIDENCE LEDGER

|

v

STATUS / TELEMETRY / RECOVERY

3. Relationship model

The “Alfred” metaphor is not a software primitive. The engineering primitive is persistent operational relationship:

Human Principal → Identity → Authority → Mission → Agent → Tool Scope → Execution → Telemetry → Evidence → Recovery.

Historical familiarity or memory is never equivalent to unlimited authority.

4. Runtime invariants

I-001 No permit -> no execution

I-002 Expired permit -> no execution

I-003 Revoked permit -> no execution

I-004 Unknown tool -> no execution

I-005 Out-of-scope resource -> no execution

I-006 Watchdog deviation -> containment

I-007 Agent cannot grant/regrant its own authority

I-008 Authorization decision -> evidence

I-009 Execution -> evidence

I-010 Evidence sequence -> hash linked

I-011 Restart preserves revocation state

I-012 Network loss does not create authority

I-013 Recovery requires authorized recovery action

5. Asynchronous operating model

A task is admitted with an explicit permit containing principal, agent, objective, capabilities, resources, temporal constraints, escalation conditions and revocation conditions.

Once admitted, the worker can continue within that envelope without requiring the human to remain at the keyboard.

The runtime emits compact state such as task status, last action, permit status, connectivity, watchdog state, exceptions and pending approvals.

If the next action cannot be established as authorized, that action stops and the reason becomes evidence.

6. Local-first model

The existing RMG architecture describes a deterministic governance kernel with zero LLM dependency, local state, SQLite/WAL, local models, a handheld/root governance node and distributed worker nodes using signed state deltas.

The first prototype deliberately reduces this to one workstation, one agent, one local governance service, one state store, one harmless local tool, one watchdog and one evidence ledger.

Mobile/root-node control and distributed synchronization are later milestones.

7. State machine

CREATED → PROPOSED → AUTHORIZED → EXECUTING → COMPLETED → EVIDENCED

At any point after authorization:

AUTHORIZED/EXECUTING → ESCALATED

AUTHORIZED/EXECUTING → CONTAINED → RECOVERY_PENDING → RECOVERED

REVOCATION is terminal for the current permit until a separate authorized recovery action occurs.

8. Action lifecycle

REQUEST → IDENTIFY → LOAD PERMIT → POLICY EVALUATION

→ DENY + EVIDENCE

or

→ PROPOSE TOOL ACTION → WATCHDOG CHECK → EXECUTE

→ VERIFY → HASH EVENT → WRITE EVIDENCE → UPDATE STATE

→ CONTINUE / COMPLETE / ESCALATE

9. Independent containment

The agent must not own the authority that determines whether it remains authorized.

Containment is scoped to the governed execution domain. The prototype must not implement a generic “shut down the entire computer/network” function.

Permitted containment primitives include task pause, tool revocation, permit invalidation, worker quarantine and recovery escalation.

10. Evidence

Each event should carry event ID, sequence, UTC timestamp, principal ID, agent ID, permit ID, action, resource, decision, result, previous-state hash and event hash.

SHA-256 provides tamper evidence for the recorded bytes; it does not independently prove authorship, origin or external truth.

11. Model abstraction

Agent Runtime → Model Adapter → Local Model / Cloud Model / Test Stub.

Model providers are replaceable. Governance remains persistent.

12. Prototype acceptance tests

1. Authorized action succeeds.
2. Unauthorized action is denied.
3. Expired permit is denied.
4. Wrong identity is denied.

5. Wrong resource is denied.
6. Out-of-scope tool is denied.
7. Runtime deviation causes containment.
8. Revocation blocks subsequent execution.
9. Agent cannot self-reauthorize.
10. Evidence hashes verify.
11. Event sequence is replayable.
12. Restart preserves revoked state.
13. Connectivity loss creates no new authority.
14. Recovery requires explicit authorized recovery.

13. Repository structure

rmg-async-alfred/

■■■■ README.md

■■■■ ARCHITECTURE.md

■■■■ pyproject.toml

■■■■ config/policy.json

■■■■ src/rmg_alfred/

■ ■■■■ models.py

■ ■■■■ permit.py

■ ■■■■ policy.py

■ ■■■■ state.py

■ ■■■■ evidence.py

■ ■■■■ watchdog.py

■ ■■■■ failsafe.py

■ ■■■■ runtime.py

■ ■■■■ transport.py

■ ■■■■ model_adapter.py

■■■■ tools/local_tool.py

■■■■ tests/

14. Build sequence

Phase 0: freeze evidence and hashes.

Phase 1: deterministic control plane.

Phase 2: one harmless local tool.

Phase 3: asynchronous worker and persistent state.

Phase 4: watchdog, revocation and scoped containment.

Phase 5: local model adapter.

Phase 6: evidence replay.

Phase 7: mobile/root control interface.

Phase 8: signed distributed state synchronization.

15. Claim discipline

Public material must distinguish architecture proposed, prototype implemented, test demonstrated, production deployment and independent validation.

The v0.1 build does not claim that RogueOS is a complete operating system, production-ready, unique, or independently validated.

It proves the narrower engineering proposition:

An autonomous worker can continue authorized work asynchronously while an independent deterministic control plane retains authority over what it may do, observes whether it remains within scope, records state transitions, and can revoke or contain execution without relying on the agent's own judgment.

16. 80/20 development loop

LEARN → BUILD → BREAK → FIX → TEST → DOCUMENT

Every meaningful build should generate code, tests, failure analysis, an architecture decision record, evidence, portfolio material and an interview story.